INTELIGENCIA ARTIFICIAL CON PYTHON SEMANA 2: *REGRESIÓN SIMPLE*

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

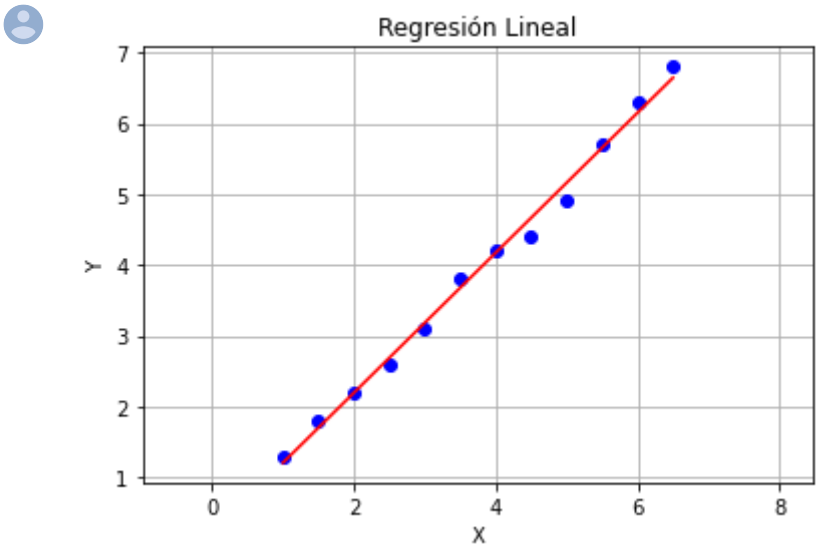

x=np.array([1,1.5,2,2.5,3,3.5,4,4.5,5,5.5,6,6.5])
y=np.array([1.3,1.8,2.2,2.6,3.1,3.8,4.2,4.4,4.9,5.7,6.3,6.8])


#Modelo de Regresión Lineal usando Mínimos Cuadrados
from sklearn import linear_model

n=x.shape[0]
xs=x.reshape((n,1))
ys=y.reshape((n,1))

modelo_rl=linear_model.LinearRegression()
modelo_rl.fit(xs,ys)
y_pred=modelo_rl.predict(xs)

plt.plot(x,y,"o",color="b")
plt.plot(x,y_pred,"-",color="r")
plt.grid()
plt.axis("equal")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Regresión Lineal")
plt.show()
```




```
print(modelo_rl.coef_)
print(modelo_rl.intercept_)
```

```
[[0.98811189]]
[0.21958042]
```

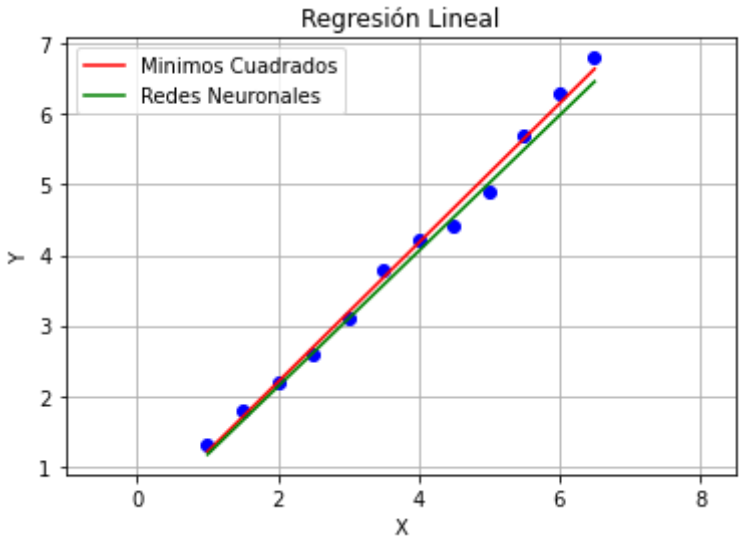
```
#Modelo usando una neurona artificial

modelo_nn=tf.keras.Sequential([tf.keras.layers.Dense(units=1,input_shape=[1])])
modelo_nn.compile(optimizer="sgd",loss="mean_squared_error")
modelo_nn.fit(x,y,epochs=8)
```

```
Epoch 1/8
1/1 [==============================] - 1s 571ms/step - loss: 16.9011
Epoch 2/8
1/1 [==============================] - 0s 13ms/step - loss: 6.9885
Epoch 3/8
1/1 [==============================] - 0s 28ms/step - loss: 2.8965
Epoch 4/8
1/1 [==============================] - 0s 22ms/step - loss: 1.2073
Epoch 5/8
1/1 [==============================] - 0s 7ms/step - loss: 0.5100
Epoch 6/8
1/1 [==============================] - 0s 7ms/step - loss: 0.2222
Epoch 7/8
1/1 [==============================] - 0s 17ms/step - loss: 0.1033
Epoch 8/8
1/1 [==============================] - 0s 7ms/step - loss: 0.0543
<keras.callbacks.History at 0x7fd42b993ad0>
```

```
y_pred_nn=modelo_nn.predict(x)

plt.plot(x,y,"o",color="b")
plt.plot(x,y_pred,"-",color="r",label="Minimos Cuadrados")
plt.plot(x,y_pred_nn,"-",color="g",label="Redes Neuronales")
plt.legend()
plt.grid()
plt.axis("equal")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Regresión Lineal")
plt.show()
```



```
modelo_nn.get_weights()
```

```
[array([[0.9624599]], dtype=float32), array([0.20492387], dtype=float32)]
```

```
#Prediccion

x_new=np.array([[7.0]])

yp_rl=modelo_rl.predict(x_new)
yp_nn=modelo_nn.predict(x_new)

print("La prediccion para {} con mínimos cuadrados es: {}".format(x_new,yp_rl))
print("La prediccion para {} con una neurona artificial es: {}".format(x_new,yp_nn))
```

```
La prediccion para [[7.]] con mínimos cuadrados es: [[7.13636364]]
La prediccion para [[7.]] con una neurona artificial es: [[6.9421434]]
```